Table of Contents

React Native 1
JSX Syntax 1
Rules of JSX 1
Creating the App 2
React Native Router Introduction 3
Tip: When Emulator Not Refreshing 6
Styling 6
Images 9
Navigating Between Screens 11
Layout File 15
Stack Component 16
Customizing Stack Screens 17
Passing Parameters to a Child Screen 20

React Native

React Native by Meta is one of the more popular code frameworks for cross platform mobile development. Having some familiarity with how it works may come in handy for your career. I have chosen the Expo implementation of React Native for this course because it is one of the simplest platforms to set up. Expo includes features like camera access, notifications, icons, image picking, and more without needing to write native code.

JSX Syntax

JSX is React syntax that lets you write HTML-like code inside JavaScript to describe what your UI should look like.

```
<View>
  <Text>Hello!</Text>
</View>
```

JSX is not real HTML. Instead, JSX offers a friendly way to create nested components. Underneath, JSX is converted into JavaScript function calls that create React Native components.

Rules of JSX

The seven rules for JSX are:

1. JSX must return a single parent element.

Valid:	Not Valid:

`return (`
`  <View>`
`  <Text>Hello</Text>`
`  </View>`
`);`
	`return (`
	`  <Text>Hello</Text>`
	`  <Text>World</Text>`
	`);`

2. Components must start with capital letters.

Valid:	Not Valid:
`<MyComponent />`	`<mycomponent />`

3. React Native Styles are applied with the style attribute:

```
<View style={{ padding: 10 }} />
```

4. Attributes are camelCase.

   Examples:

   ```
   onPress={...}
   backgroundColor="red"
   marginTop={10}
   ```

5. Variables, function calls, inline expressions go inside curly braces.
6. JSX can't return raw text outside a Text component.

Valid:	Not Valid:
`<Text>Hello</Text>`	`<View>Hello</View>`

7. Self-closing tags must end with />

   Examples:

   ```
   <TextInput />
   <Image />
   ```

Creating the App

Example 1: Create the app

To create the app, make a folder where you have full permissions. Then open the folder in VS Code. Launch an Android emulator from Android Studio and wait for it to launch so the home screen appears. Next, open the VS Code terminal by navigating to **Terminal | New Terminal**. Then cd into the folder and run this command to create the app:

```
npx create-expo-app@latest --template blank

(prefix this with sudo on a Mac)
```

I experienced several permissions issues on my Mac. In terminal I had to grant read write permission on my project folder and sub folders:

```
sudo chown -R $USER ./
```

To start the app, in the same folder as package.json run:

```
npx expo start
```

Be sure to press **a** to run the app in the Android emulator which should already be running.

React Native Router Introduction

The style of expo routing with index.jsx placed inside an app folder as the app entry point will be used for all React Native examples which are covered from this point onwards in this course.

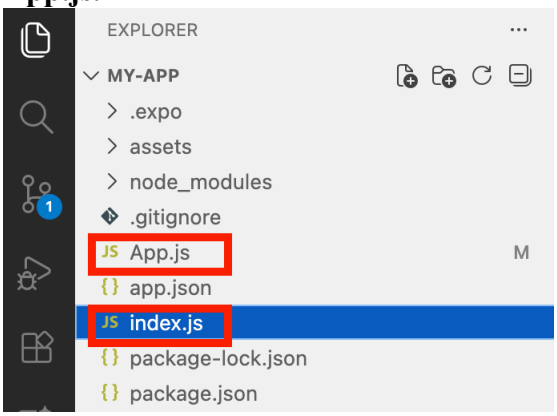
Example 2: Customizing the App

This example will make adjustments to use the React Native router. To enable routing install the following dependencies. The most current collection of required dependencies are documented at https://docs.expo.dev/router/installation/#manual-installation

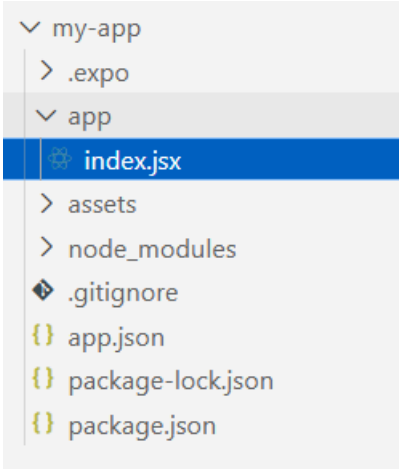
Run this command in the VS Code terminal to get the dependences:

```
npx expo install expo-router react-native-safe-area-context react-native-screens expo-linking
expo-constants expo-status-bar
```

Then, in your project folder delete the app entry point which is defined **index.js**. Also delete **App.js**.



Next create an app folder and add a new **index.jsx** file in the app folder. Be sure to use lower case.



The code inside **index.jsx** defines a component called Home. The Home component is actually made up of other components including View and Text. The tags to implement these components are not actually HTML tags – they are React Native components which are designed to run in iOS and Android environments. The native components must be imported so they can be used.

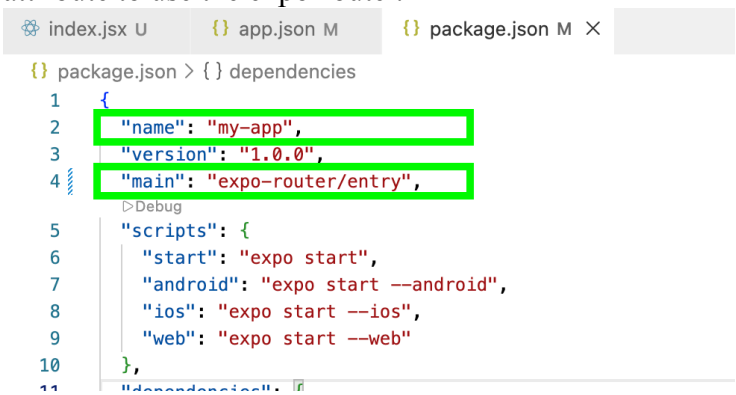
app\index.jsx

```
import { StyleSheet, Text, View } from 'react-native'
import React from 'react'

const Home = () => {
  return (
    <View>
      <Text>This is my home screen</Text>
    </View>
  )
}
export default Home
const styles = StyleSheet.create({})
```

Next go to **app.json** and add a scheme which allows for linking from an external source.

In **package.json**, ensure the name matches the name in the app.json file. Change the *main* attribute to use the expo-router.



When running the app, the output appears but it is formatted badly. The unformatted text will appear at the top left and may barely be visible due to the lack of formatting.

Tip: You may need to press r to refresh the app when running it to have the new routing take effect.

Exercise 1 (1 mark)

Why would this syntax cause an error?

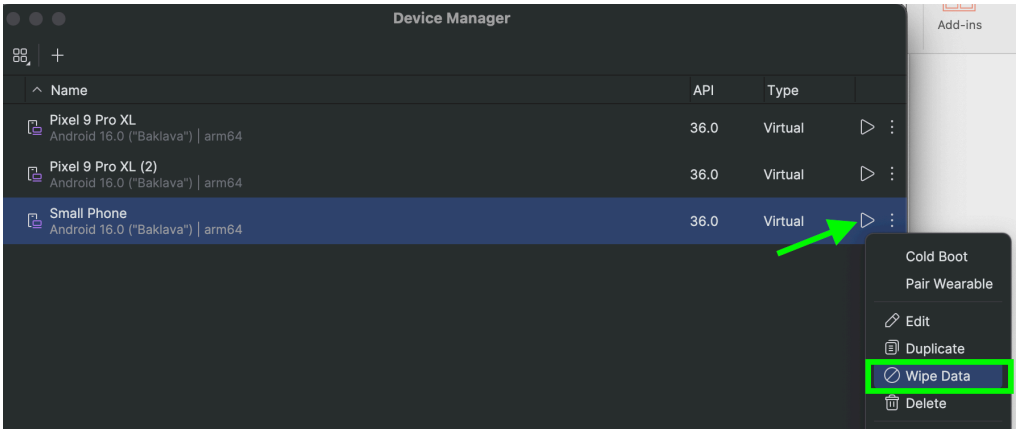
```
<View>
  Hello
  <Text>This is my home screen</Text>
</View>
```

because "Hello" is raw text sitting directly inside `<View>`

Tip: When Emulator Not Refreshing

When working on React Native apps, you may find that the app occasionally does not refresh when pressing **r** in the VS Code terminal. There may be better methods to handle this situation but currently these steps work well for me:

1. Stop the app from running.
2. Shut the emulator down.
3. Launch the Virtual Device Manager from Android Studio, wipe the emulator you want to use and then restart the emulator.

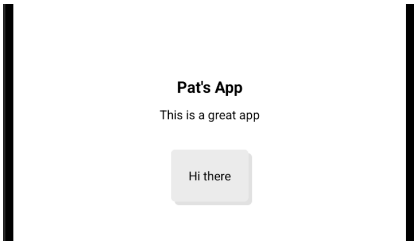


4. After the emulator has finished launching and the home screen is visible, relaunch React Native app.
5. If the emulator still does not to refresh, try using simple code that has been proven to work such as the code for Example 2.

Styling

Example 3: Adding Styling

This example demonstrates how to add some basic styles.



To enable the styling, a StyleSheet will be modified to define style properties for alignment, padding, fonts and more. The StyleSheet provides a structure for formatting which is a lot like CSS.

app\index.jsx

```
import { StyleSheet, Text, View } from 'react-native'

const Home = () => {
    return (
        <View style={styles.container}>
            <Text style={styles.title}>Pat's App</Text>
            <Text style={{ marginTop: 10, marginBottom: 30 }}>
                This is a great app
            </Text>
            <View style={styles.card}>
                <Text>Hi there</Text>
            </View>
        </View>
    )
}

export default Home

const styles = StyleSheet.create({
    container: {
        flex: 1,
        alignItems: 'center',
        justifyContent: 'center'
    },
    title: {
        fontWeight: 'bold',
        fontSize: 18
    },
    card: {
        backgroundColor: '#eee',
        padding: 20,
        borderRadius: 5,
        boxShadow: '4px 4px rgba(0,0,0,0.1)'
    }
});
```

Exercise 2 (2 marks)

Starting with Example 2, create a customMargin property in the styles object so the format has a marginTop of 10 and a marginBottom of 30. Then replace modify the inline style definition;

```
<Text style={{ marginTop: 10, marginBottom: 30 }}>
```

with a reference to the customMargin property. Show your revised styles object here:

```
const styles = StyleSheet.create({
    container: {
        flex: 1,
        alignItems: 'center',
        justifyContent: 'center'
    },
    title: {
        fontWeight: 'bold',
        fontSize: 18
    },
    card: {
        backgroundColor: '#eee',
        padding: 20,
        borderRadius: 5,
        boxShadow: '4px 4px rgba(0,0,0,0.1)'
    },

    customMargin: {
       marginTop: 10,
       marginBottom: 30,
    },
});
```

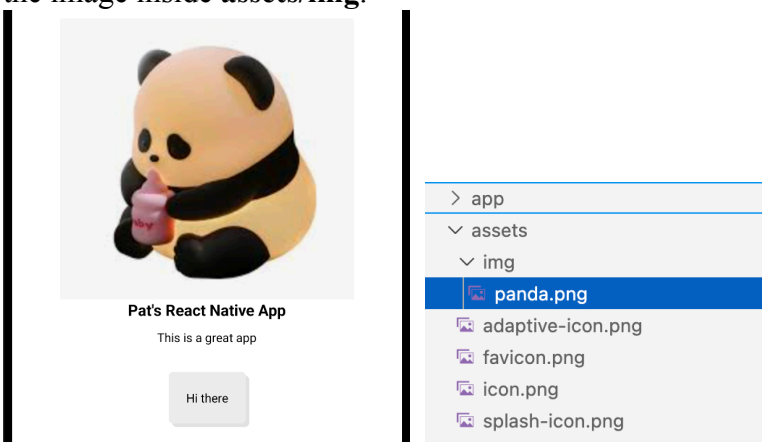
Show your revised Text component here once it references the new *styles* object property.

```
<Text style={styles.customMargin }>
```

Images

Example 4: Adding Images

This example demonstrates how to modify Example 3 so it displays an image with the image component. Note that images should be placed within the assets directory. For my case, I placed the image inside **assets/img**.



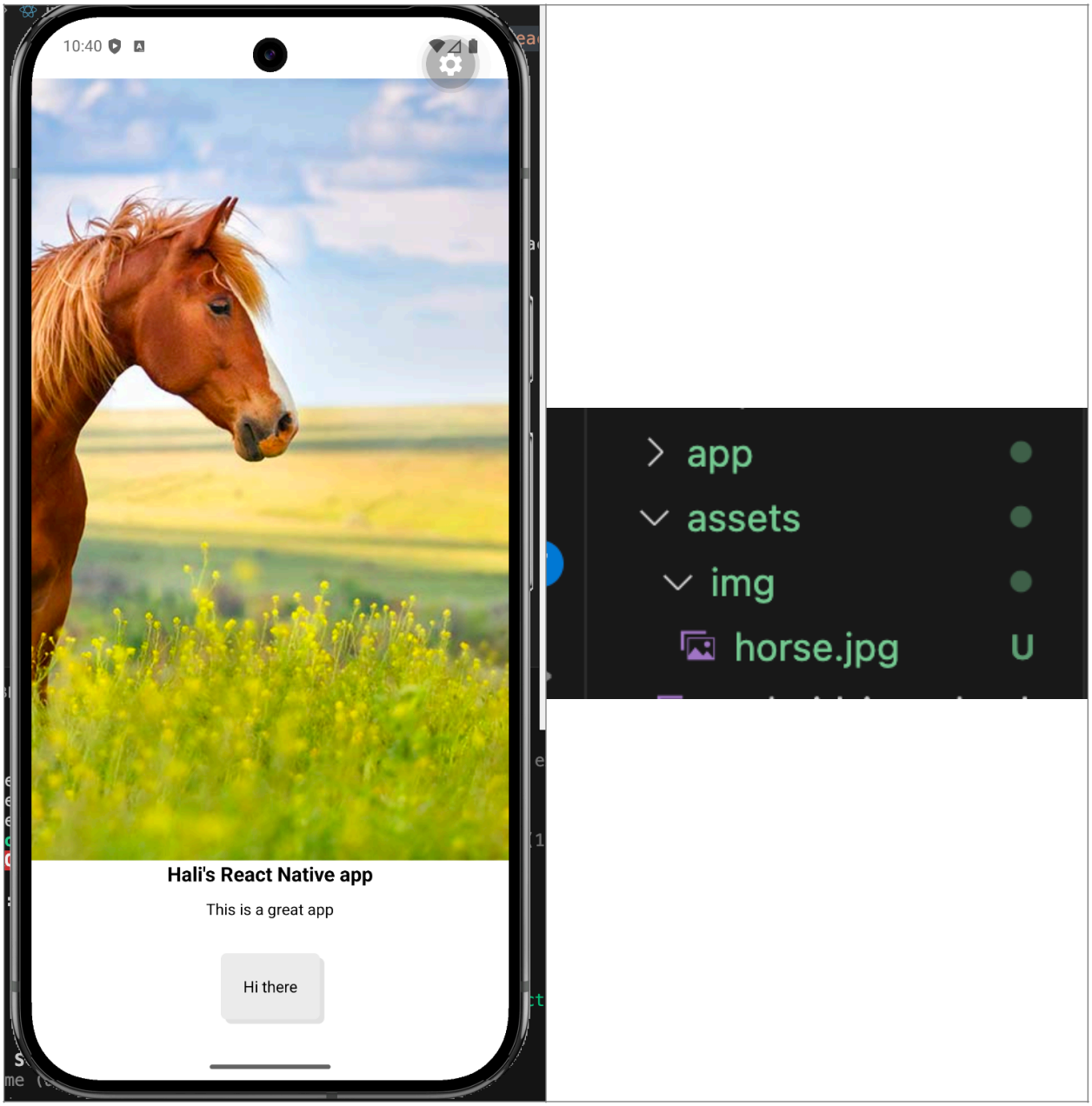
The new code to enable this change is highlighted below:

```
import { StyleSheet, Text, View, Image } from 'react-native'
import Logo from '../assets/img/panda.png'

const Home = () => {
    return (
        <View style={styles.container}>
              <Image source={Logo}></Image>
```

Exercise 3 (2 marks)

Modify the app so it says “Your first name React Native app” and so it shows an image of an animal that has the same first letter as your name. Show a screenshot of the output after making this change:



Navigating Between Screens

Example 5: Navigating between screens.

This example shows how to create an app with a Home screen which can access an About screen with the help of a **Link** component.

To build this, starting with Example 4, add the following about.jsx file to the app folder.

app/about.jsx

```
import { StyleSheet, Text, View } from 'react-native'

const About = () => {
  return (
    <View style={styles.container}>
      <Text style={styles.title}>About Page</Text>
    </View>
  )
}

export default About

const styles = StyleSheet.create({
  container: {
    flex: 1,
    alignItems: 'center',
    justifyContent: 'center'
  },
  title: {
    fontWeight: 'bold',
    fontSize: 18
  },
})
```

Then replace the index.jsx file with this version. The Link to the About screen is highlighted below.

app/index.jsx

```
import { StyleSheet, Text, View } from 'react-native'
import { Link } from 'expo-router'

const Home = () => {
  return (
      // flex: 1 centers screen vertically so to occupy entire screen.
      <View style={{ flex: 1, justifyContent: 'center' , padding: 40}}>
        {/* right aligns content */}
        <View style={{ flexDirection: 'row', justifyContent: 'flex-end' }}>
          <Link href="/about" style={styles.navlink}>About</Link>
        </View>
        <View style={styles.container}>
          <Text style={styles.title}>Pat's App</Text>
          <Text style={{ marginTop: 10, marginBottom: 30 }}>
            This is a great app
          </Text>
        </View>
      </View>
  )
}

export default Home

const styles = StyleSheet.create({
  container: {
    alignItems: 'center',
    justifyContent: 'center'
  },
  title: {
    fontWeight: 'bold',
    fontSize: 18
  },
  navlink: {
    color: 'blue',
    textDecorationLine: 'underline',
    fontSize: 20
  }
});
```

Exercise 4 (1 mark)

The two comment styles below are required in different situations. Both comment styles are used in Example 5. For react native, when is the following comment style appropriate?

```
// this is a comment.
```

used **outside** JSX, in regular JavaScript code

For react native, when is the following comment style appropriate?

```
{/* this is a comment. */}
```

used **inside** JSX elements

Exercise 5 (1 mark)

Fill in the blanks with attributes that are used in Example 5.

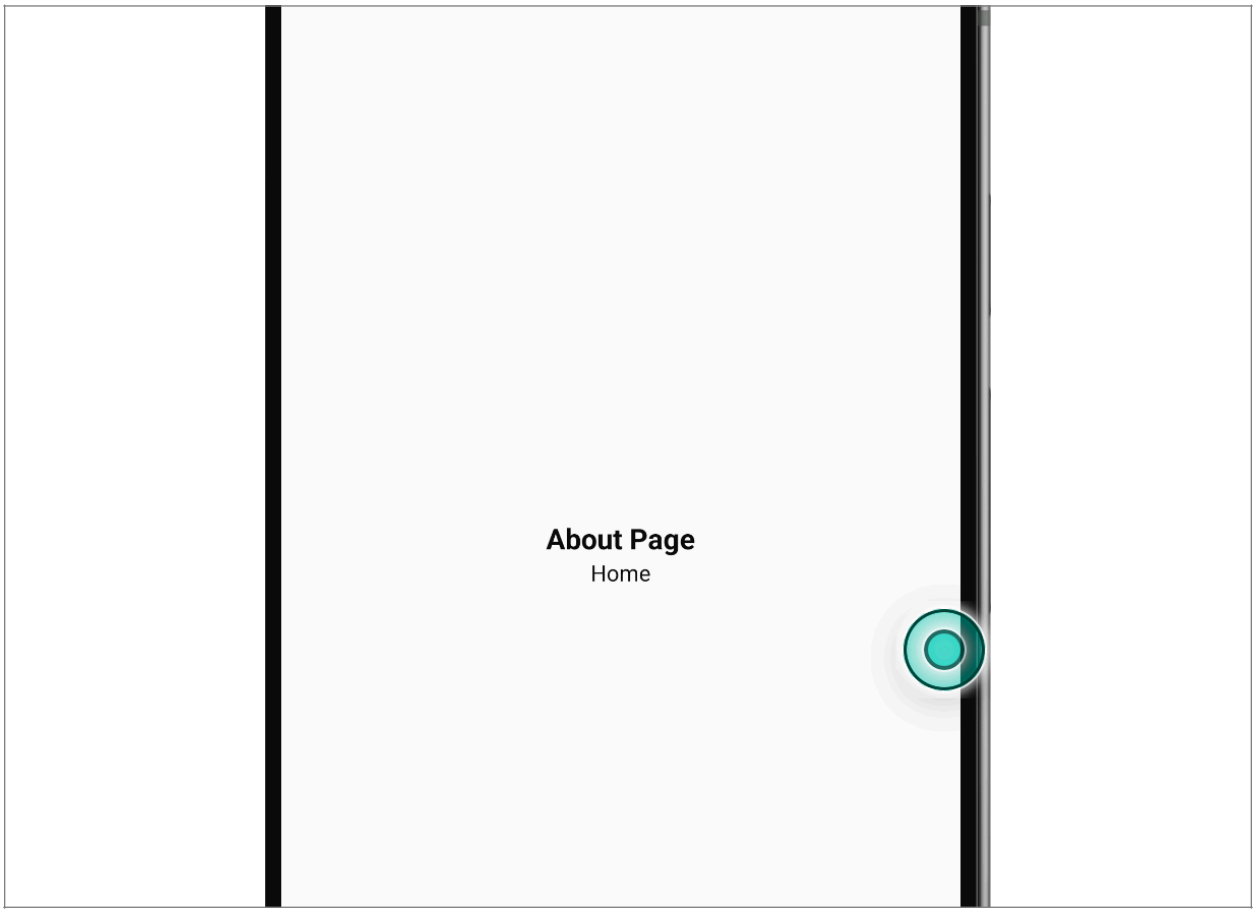
justifyContent centers content vertically.

alignItems right aligns content.

padding adds padding about a component.

Exercise 6 (2 marks)

On the About screen, add a link to return to the Home screen. Show a screenshot of your About screen once this link has been added.

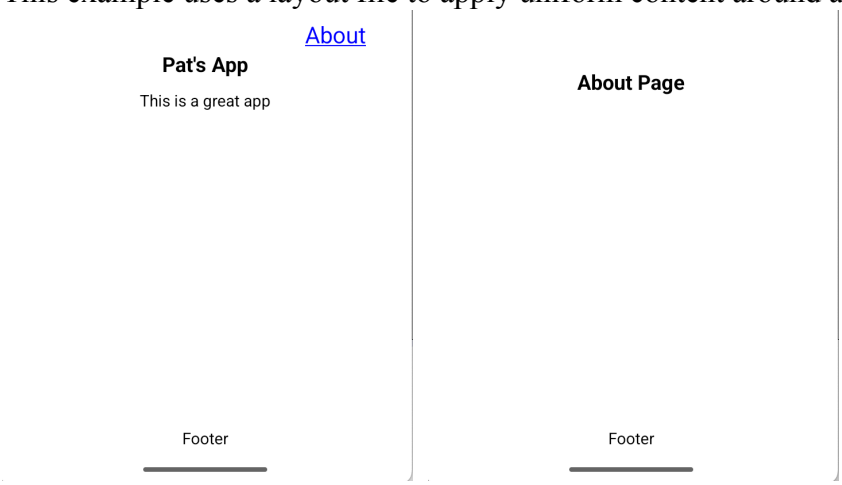


Layout File

The _layout.jsx defines the navigation structure for a folder in an Expo Router project. It tells Expo how to display the screens inside that folder—such as using tabs or stacks and applies shared configuration (headers, themes, screen options) to all child screens.

Example 6: Adding a Layout

This example uses a layout file to apply uniform content around all screens.



The name must be _layout.jsx. Expo router looks for _layout.jsx and loads index.jsx by default.
app_layout.jsx

```
import { StyleSheet, Text, View,  } from 'react-native'
import { Slot } from 'expo-router'

const RootLayout = () => {
  return (
    <View style={{ flex: 1 }}>

     {/* placeholder for screen */}
      <Slot/>
      {/* alignmItems only works in containers - not Text */}
      <View style={{alignItems: 'center'}}>
      <Text style={{  marginTop: 10, marginBottom: 30, }}>Footer</Text>
      </View>
    </View>
  )
}
export default RootLayout
const styles = StyleSheet.create({})
```

Exercise 7 (1 mark)

The *alignItems* only works for containers but not for text. However, the `textAlign:'center'` style works for a Text component. Remove the centering container in _layout.jsx and instead use textAlign. Show your revised _layout.jsx code here after verifying that it works properly:

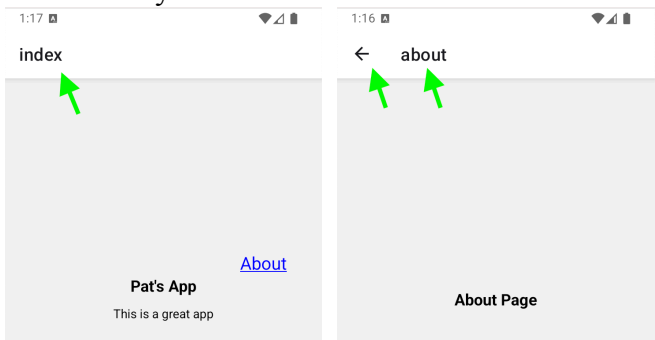
```
7
8     {/* placeholder for screen */}
9       <Slot/>
10      {/* alignmItems only works in containers - not Text */}
11      <View>
12      <Text style={{ marginTop: 10, marginBottom: 30, textAlign: 'center'}}
13      </View>
14    </View>
15  )
16
```

Stack Component

The Stack component from expo-router creates a stack navigator for all routes inside the folder it's placed in. It also automatically managers header titles, back buttons, transitions, and navigation history for you.

Example 7: Stack

This example demonstrates how to use the Stack component to enable navigation. Note how it automatically adds screen titles and a back button.



To build this example starting with Example 6, replace the references to the Slot component with Stack as shown below.

app_layout.jsx

```
import { StyleSheet, Text, View,  } from 'react-native'
import { Stack } from 'expo-router'

const RootLayout = () => {
  return (
    <View style={{ flex: 1 }}>
      <Stack/>
      {/* alignmItems only works in containers - not Text */}
      <Text style={{  marginTop: 10, marginBottom: 30, textAlign: 'center'}}
>Footers</Text>

    </View>
  )
}

export default RootLayout
const styles = StyleSheet.create({})
```

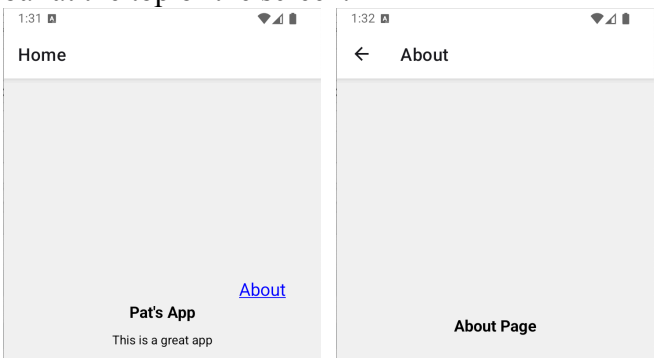
Customizing Stack Screens

We can implement a more customizable Stack with the Stack.Screen component.

```
<Stack.Screen name="index" options={{ title: 'Home' }} />
```

Example 8: Customizing Stack Screens

This example demonstrates how Stack.Screen can be used to create a customized screen title. Note also that the Stack component also automatically centers the content vertically with a title bar at the top of the screen.



To build this example, replace _layout.jsx from Example 7 with this version.

```
import { Stack } from 'expo-router'
import { StyleSheet } from 'react-native'

const RootLayout = () => {
  return (
    <Stack>
      <Stack.Screen name="index" options={{ title: 'Home' }} />
      <Stack.Screen name="about" options={{ title: 'About' }} />
    </Stack>
  )
}
export default RootLayout
const styles = StyleSheet.create({})
```

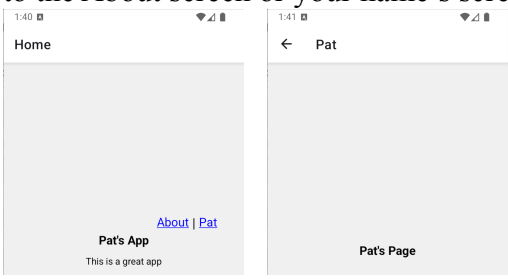
Exercise 8 (1 mark)

Compared to Example 7, Example 8 does not use `<View style={{ flex: 1 }}>` but it achieves a similar format. How is this possible?

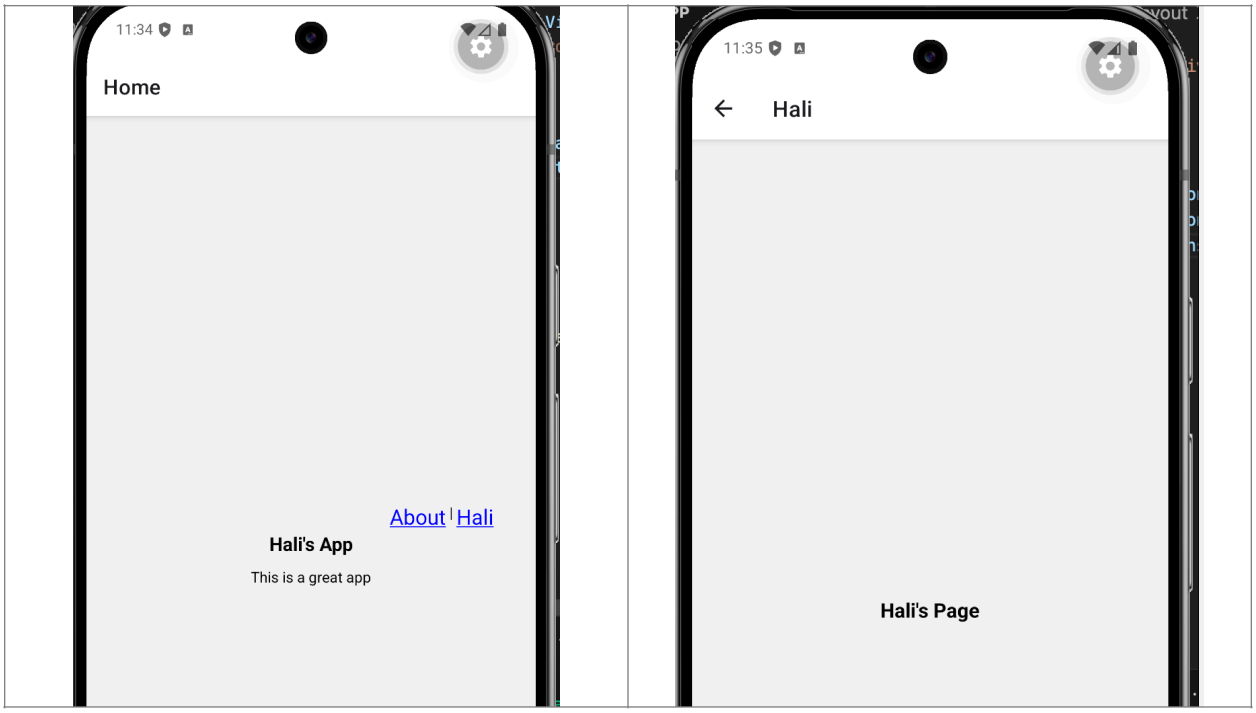
Stack component automatically fills the full screen, so there's no need to manually set flex: 1 on a View!

Exercise 9 (3 marks)

Add a third screen with your name as the title. Change the home screen so it can navigate either to the About screen or your name's screen depending on the user's selection.

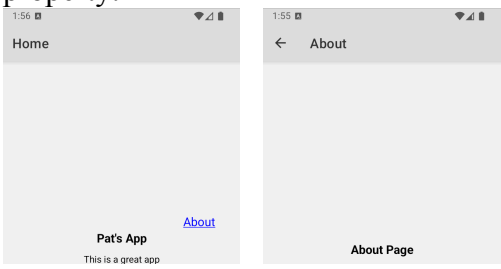


Show screenshots of your Home screen and name screen (with your name in it here) here:



Example 9: Customizing Stack Screens Continued

This example demonstrates how to apply global settings for stack screens with the screenOptions property:



To build this, replace the RootLayout component in Example 8 with this version.

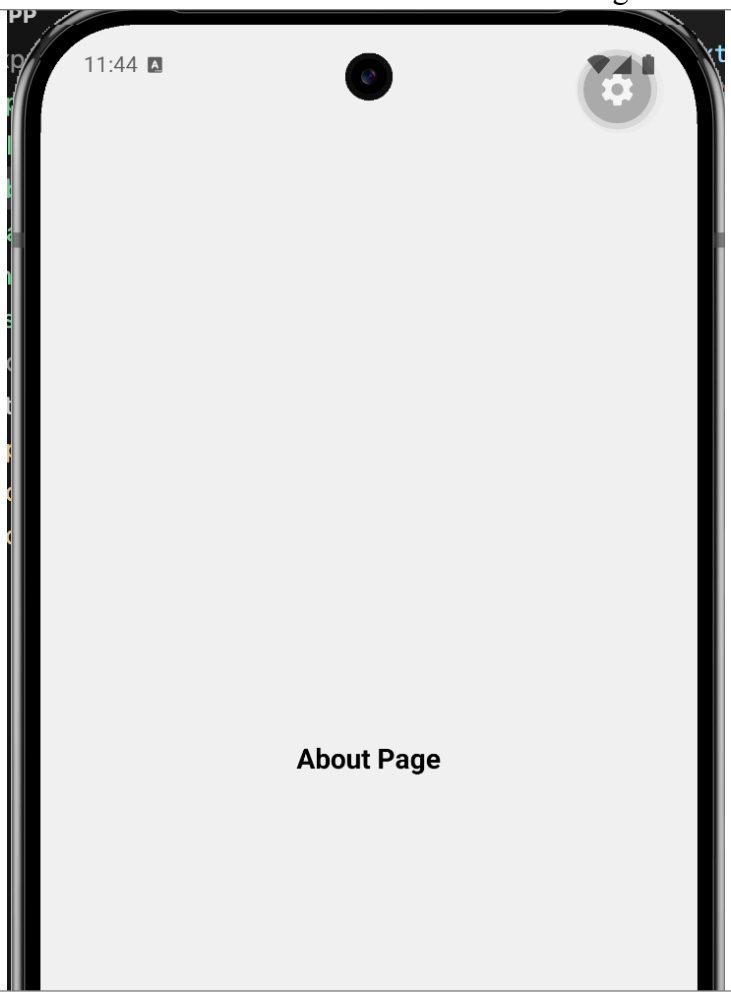
```
const RootLayout = () => {
  return (
    <Stack screenOptions={{
      headerStyle: { backgroundColor: '#ddd' },
      headerTintColor: '#333',
    }}>
      <Stack.Screen name="index" options={{ title: 'Home' }} />
      <Stack.Screen name="about" options={{ title: 'About' }} />
    </Stack>
  )
}
```

Exercise 10 (2 marks)

Make the stack header the color yellow and then replace the Stack.Screen component with this version for the About screen. This will remove the header from the About screen:

```
<Stack.Screen name="about" options={{ title: 'About', headerShown: false  }} />
```

Show screenshots of the Home screen and About screen after making these changes.



Passing Parameters to a Child Screen

Example 10: Passing Parameters to a Child Screen

This example demonstrates how to pass parameters to a child screen from the parent screen. To begin, start with Example 9. Then, in **index.jsx** add these parameters to the Link element.

app\index.jsx

```
    params: { fname: "Pat", courseNum:3717 }}}
```

This **about.jsx** code uses the useLocalSeachParams hook to retrieve the parameters.

app\about.jsx

```
import { View, Text } from 'react-native';
import { useLocalSearchParams } from 'expo-router';

export default function About() {
  const { fname, courseNum } = useLocalSearchParams();

  return (
    <View style={{ flex: 1, justifyContent: 'center', alignItems: 'center' }}>
    <Text>First Name: {fname}</Text>
    <Text>Course Num: {courseNum}</Text>
    </View>
  );
}
```

Exercise 11 (2 marks)

Modify Example 10 to pass your first name, last name and course number as separate parameters and show them in the about screen. Show your code in index.jsx which passes the three parameters to the About screen.

11:56
First Name: Hali
Last Name: Imanpanah
Course Num: 3717

Show the revised code in about.jsx here:

```
import { View, Text } from 'react-native';
import { useLocalSearchParams } from 'expo-router';

export default function About() {
  const { fname, lname, courseNum } = useLocalSearchParams();

  return (
    <View style={{ flex: 1, justifyContent: 'center', alignItems:
'center' }}>
    <Text>First Name: {fname}</Text>
    <Text>Last Name: {lname}</Text>
    <Text>Course Num: {courseNum}</Text>
    </View>
  );
}
```